

## th:object et la syntaxe astérisque \*{ }

### 1. La syntaxe astérisque \*{ }

Dans Thymeleaf, une expression de variable s'écrit `${...}`. Il existe une seconde forme, très utilisée : `*{...}`, la **syntaxe astérisque** (*selection expression*).

La différence tient au point de départ de l'évaluation :

| Syntaxe              | Évaluée sur                                             |
|----------------------|---------------------------------------------------------|
| <code>\${...}</code> | L'ensemble du contexte (toutes les variables du modèle) |
| <code>*{...}</code>  | L' <b>objet sélectionné</b> par <code>th:object</code>  |

Pour sélectionner un objet, on utilise l'attribut `th:object`. L'objet reste sélectionné pour l'élément qui porte l'attribut **et tous ses descendants**.

#### Premier exemple

Syntaxe astérisque :

```
<div th:object="${person}" class="box">
  <p><b>Nom complet :</b> <span th:text="*{fullName}"></span></p>
  <p><b>E-mail :</b> <span th:text="*{email}"></span></p>
</div>
```

C'est exactement équivalent à la syntaxe dollar :

```
<div class="box">
  <p><b>Nom complet :</b> <span th:text="${person.fullName}"></span></p>
  <p><b>E-mail :</b> <span th:text="${person.email}"></span></p>
</div>
```

L'intérêt apparaît dès que l'on accède à plusieurs propriétés du même objet : on n'a plus à répéter le nom `person`, et le code devient plus court et plus facile à modifier.

#### Mélanger les deux syntaxes

On peut mélanger `*{...}` et `${...}` dans la même zone. Dans une zone où un objet est sélectionné, `${...}` continue de s'évaluer sur **tout le contexte**, et non sur l'objet sélectionné.

```
<div th:object="${person}" class="box">
  <p><b>Nom complet :</b> <span th:text="*{fullName}"></span></p>
  <p><b>E-mail :</b> <span th:text="${person.email}"></span></p>
</div>
```

Ici, `*{fullName}` lit la propriété `fullName` de l'objet sélectionné, et `${person.email}` lit la propriété `email` de la variable `person` du contexte. Les deux désignent le même objet parce que `th:object` a sélectionné `${person}` ; mais la seconde écriture passe par le nom de la variable et ne dépend pas de la sélection.

L'objet sélectionné lui-même est accessible par `${#object}`, ce qui permet par exemple d'écrire `${#object.fullName}` (équivalent à `*{fullName}`).

## Sans objet sélectionné

Lorsqu'aucun objet n'est sélectionné, `*{...}` s'évalue sur tout le contexte : `${...}` et `*{...}` sont alors équivalents.

```
<div class="box">
  <p><b>Nom complet :</b> <span th:text="*{person.fullName}"></span></p>
  <p><b>E-mail :</b> <span th:text="*{person.email}"></span></p>
</div>

<!--/* Équivalent à : */-->
<div class="box">
  <p><b>Nom complet :</b> <span th:text="${person.fullName}"></span></p>
  <p><b>E-mail :</b> <span th:text="${person.email}"></span></p>
</div>
```

## Exemple de modèle et de contrôleur

```
public class Person {

    private final String fullName;
    private final String email;

    public Person(String fullName, String email) {
        this.fullName = fullName;
        this.email = email;
    }

    public String getFullName() { return fullName; }
    public String getEmail() { return email; }
}

@GetMapping("/object-example")
public String objectExample(Model model) {
    model.addAttribute("person", new Person("Tom", "tom@example.com"));
    return "object-example";
}
```

## 2. Points à connaître

### Portée : l'élément et ses descendants

La sélection ne s'applique qu'à l'élément portant `th:object` et à son contenu. En dehors de cet élément, `*{...}` retombe sur l'ensemble du contexte. Les sélections peuvent être imbriquées : l'objet le plus interne s'applique dans sa zone.

### `th:object` dans une boucle

Sur une même balise, `th:each` est évalué **avant** `th:object` : le `th:object` peut donc utiliser la variable de boucle.

```
<tr th:each="p : ${people}" th:object="${p}">
  <td th:text="*{fullName}">Nom</td>
  <td th:text="*{email}">E-mail</td>
</tr>
```

### `th:object` et `th:with`

| Attribut               | Rôle                                                                                                       |
|------------------------|------------------------------------------------------------------------------------------------------------|
| <code>th:object</code> | Définit l' <b>objet sélectionné</b> , accessible par <code>*{...}</code> ; un seul à la fois dans une zone |
| <code>th:with</code>   | Définit des <b>variables locales nommées</b> (une ou plusieurs), accessibles par <code>\${...}</code>      |

### Accès sécurisé aux propriétés imbriquées

Avec SpringEL, l'opérateur `?.` évite une exception si un objet intermédiaire est null :

```
<span th:text="*{address?.city} ?: 'Ville inconnue'">Ville</span>
```

## 3. Formulaires Spring : `th:object` et `th:field`

Dans un formulaire Spring MVC, la syntaxe astérisque est omniprésente : `th:object` désigne l'objet formulaire (*form-backing object*) et `th:field="*{...}"` lie chaque champ à l'une de ses propriétés.

```
<form th:action="@{/register}" th:object="${appUserForm}" method="post">

  <label for="userName">Nom d'utilisateur </label>
  <input type="text" th:field="*{userName}" />
  <br/>

  <label for="password">Mot de passe </label>
  <input type="password" th:field="*{password}" autocomplete="new-password"
/>
  <br/>
```

```
<label for="confirmPassword">Confirmation :</label>
<input type="password" th:field="*{confirmPassword}" autocomplete="new-
password" />
<br/>

<label for="email">E-mail :</label>
<input type="email" th:field="*{email}" />
<br/>

<input type="submit" value="Valider" />
</form>
```

Ce que fait `th:field="*{userName}"` :

- il génère les attributs `id="userName"` et `name="userName"` ;
- il remplit l'attribut `value` avec la valeur actuelle de la propriété `userName` (ce qui permet de réafficher la saisie après une erreur) ;
- il tient compte du type du champ : `text`, `email`, `checkbox`, `radio`, `<select>`, `<textarea>`...

Règles à respecter :

- `th:field` s'écrit toujours avec la syntaxe astérisque `*{...}` et doit se trouver dans un élément portant `th:object`.
- Dans un formulaire Spring, `th:object` doit désigner **uniquement le nom d'un attribut du modèle** : `th:object="${appUserForm}"` est correct, alors que `th:object="${appUserForm.address}"` (navigation dans une propriété) ne convient pas pour un formulaire lié.
- Les champs de type `password` ne sont **pas** pré-remplis lors du réaffichage : c'est volontaire, pour ne pas renvoyer un mot de passe dans le HTML.
- Si l'objet formulaire n'est pas dans le modèle au moment du rendu, Spring lève l'erreur « *Neither BindingResult nor plain target object for bean name* » (voir le document sur les listes déroulantes).

## Exemple complet : inscription avec validation

L'objet formulaire :

```
package local.epul4a.demosbt.form;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public class AppUserForm {

    @NotBlank(message = "Le nom d'utilisateur est obligatoire")
    @Size(min = 3, max = 30, message = "Entre 3 et 30 caractères")
```

```
private String userName;

@NotBlank(message = "Le mot de passe est obligatoire")
@Size(min = 8, message = "Au moins 8 caractères")
private String password;

@NotBlank(message = "La confirmation est obligatoire")
private String confirmPassword;

@NotBlank(message = "L'e-mail est obligatoire")
@email(message = "E-mail invalide")
private String email;

// getters et setters pour les quatre propriétés
}
```

Le contrôleur :

```
@Controller
public class RegisterController {

    @GetMapping("/register")
    public String showForm(Model model) {
        model.addAttribute("appUserForm", new AppUserForm());
        return "register";
    }

    @PostMapping("/register")
    public String register(@Valid @ModelAttribute("appUserForm") AppUserForm
form,
                           BindingResult result) {

        // Contrôle qui porte sur deux champs : non exprimable par une
annotation simple
        if (!result.hasFieldErrors("confirmPassword")
            && !form.getPassword().equals(form.getConfirmPassword())) {
            result.rejectValue("confirmPassword", "password.mismatch",
                "Les mots de passe ne correspondent pas");
        }

        if (result.hasErrors()) {
            return "register"; // réaffichage avec les erreurs
        }

        // ... enregistrer l'utilisateur (mot de passe haché, voir ci-
dessous)
        return "redirect:/registerSuccess"; // Post/Redirect/Get
    }
}
```

Le template, avec affichage des erreurs :

```
<form th:action="@{/register}" th:object="${appUserForm}" method="post">

  <div>
    <label for="userName">Nom d'utilisateur :</label>
    <input type="text" th:field="*{userName}" th:errorclass="is-invalid" />
    <span class="error" th:if="${#fields.hasErrors('userName')}}"
      th:errors="*{userName}">Erreur</span>
  </div>

  <div>
    <label for="password">Mot de passe :</label>
    <input type="password" th:field="*{password}" th:errorclass="is-invalid" />
    <span class="error" th:if="${#fields.hasErrors('password')}}"
      th:errors="*{password}">Erreur</span>
  </div>

  <div>
    <label for="confirmPassword">Confirmation :</label>
    <input type="password" th:field="*{confirmPassword}" th:errorclass="is-invalid" />
    <span class="error" th:if="${#fields.hasErrors('confirmPassword')}}"
      th:errors="*{confirmPassword}">Erreur</span>
  </div>

  <div>
    <label for="email">E-mail :</label>
    <input type="email" th:field="*{email}" th:errorclass="is-invalid" />
    <span class="error" th:if="${#fields.hasErrors('email')}}"
      th:errors="*{email}">Erreur</span>
  </div>

  <input type="submit" value="Valider" />
</form>
```

Les outils d'erreur utilisés :

- `#fields.hasErrors('userName')` : vrai si le champ a au moins une erreur ; les noms de champs sont relatifs à l'objet sélectionné ;
- `th:errors="*{userName}"` : affiche les messages d'erreur du champ ;
- `th:errorclass="is-invalid"` : ajoute la classe CSS indiquée à l'élément si son champ est en erreur (voir le document sur `th:classappend`).

**Sécurité.** Dans une vraie application, un mot de passe n'est **jamais** stocké en clair. On le hache avec un PasswordEncoder de Spring Security (par exemple BCryptPasswordEncoder) avant l'enregistrement. Le formulaire doit en outre être servi en HTTPS. Avec Spring Security, `th:action` ajoute automatiquement le jeton CSRF dans le formulaire.

## Résumé

- `${...}` s'évalue sur tout le contexte ; `*{...}` sur l'objet sélectionné par `th:object`, ou sur tout le contexte si aucun objet n'est sélectionné.
- La sélection vaut pour l'élément et ses descendants ; `${...}` reste utilisable dans la zone et continue de s'évaluer sur tout le contexte.
- `th:each` est évalué avant `th:object` sur une même balise.
- `th:object` sélectionne un objet pour `*{...}` ; `th:with` définit des variables locales nommées.
- Dans un formulaire Spring, `th:object` désigne un attribut du modèle et `th:field="*{...}"` lie chaque champ à une propriété ; `th:errors`, `#fields.hasErrors` et `th:errorclass` gèrent les erreurs de validation.
- Les champs password ne sont pas pré-remplis, et les mots de passe doivent être hachés.

## Exercices proposés

1. Réécrire une fiche d'utilisateur contenant six propriétés, d'abord avec `${...}`, puis avec `th:object` et `*{...}`.
2. Dans un tableau, utiliser `th:each` et `th:object` sur la même balise `<tr>` pour afficher les propriétés d'une liste de personnes.
3. Expliquer ce que produit `<div th:object="${person}"><span th:text="${fullName}"></span></div>` et pourquoi ce n'est pas équivalent à `*{fullName}`.
4. Ajouter au formulaire d'inscription un champ « date de naissance » avec une règle `@Past`, puis afficher son erreur.
5. Écrire un test `@WebMvcTest` qui soumet deux mots de passe différents et vérifie que le modèle contient une erreur sur `confirmPassword`.